

**Synligare utveckling av inbyggda fordonssystem –
visualiserad kravhantering och samverkan**

**Visibler Development of Embedded Automotive Systems -
Visualised Requirements Management and Collaboration**

Report type	Deliverable D3.1
Report name	Report on Tooling
Dissemination level	Public
Status	Initial Release
Version number	0.5
Date of preparation	2014-06-30



Authors

Editor

Jan Söderberg

E-mail

jan.soderberg@systemite.se

Authors

Mikael Stolpe

E-mail

mikael.stolpe@arccore.com

Jan Söderberg

jan.soderberg@systemite.se

Henrik Lönn

henrik.lonn@volvo.com

The Consortium

Volvo Technology Corporation AB - Autoliv AB - Arccore AB- Systemite AB - Semcon Sweden AB



Revision chart and history log

Version	Date	Reason
0.1	2014-01-01	Document skeleton
0.5	2014-06-30	Initial Release

Table of contents

Authors.....	3
Revision chart and history log.....	5
Table of contents	7
1 Introduction	9
1.1 Purpose.....	9
1.2 Scope.....	9
2 Supported Scenarios	10
3 EATOP and ARTOP	11
3.1 TableView Plug-in	11
3.2 Create Connector Plug-in	12
3.3 Error Model Synthesis.....	13
3.3.1 Description	13
4 SystemWeaver	15
4.1 Metrics Extension.....	15
5 EnterpriseArchitect.....	16
5.1 Enterprise Architect EAXML Exchange.....	16
5.1.1 Description	16
6 Summary.....	17
7 References.....	18

1 Introduction

This document is the report on results of work package 3 (WP3) of Project Synligare.

The overall effort in Project Synligare is to address specification of complex automotive embedded systems to provide an overview, ease review and management of requirements by means of defining structured information models for representing system specification, which will enable automatic generation of views and metrics. The purpose is to improve collaborative development of automotive embedded systems in comparison with the typical situation in which there is no common specification language. This involves addressing challenges like relating requirements and design with respect to configurations in a product family by applying various relations and searches in structured requirement information based on existing specification languages like EAST-ADL. Further, it involves management of specifications and requirements in accordance with ISO 26262. The aim is that the outcomes of Project Synligare should be tried on an example system, to validate the structured information models, the views and the metrics.

1.1 Purpose

The work in WP3 will result in different tool implementations, supporting the concepts and methods defined in the project [1], including technical validation on tool level. The tool implementations will be formally tried and evaluated in WP4, in scenarios emulating real industrial cases. The different tool technologies used in the implementation represents not only technologies widely in active use in the industry today for the various roles in Embedded Automotive Systems, but are also technologies well familiar within the project consortium.

1.2 Scope

The scope of WP3 is to implement tool support for the scenarios described in 2, for the tool technologies described further in this report.

2 Supported Scenarios

The implementation in EATOP, SystemWeaver and EnterpriseArchitect supports the specific use cases and scenarios described in this section. These scenarios represent a realistic project set-up including an OEM and Suppliers, and work as a basis for the practical integration and validation of the Modeling, Model Exchange, View and Metrics generation in the different tools.

The specification of these scenarios can be found in section 2.2 in [1].

A technical challenge in managing the information in distributed projects with a high degree of collaboration is that the same information needs to be represented in multiple tool environments. This means that the same information can be altered – by mistake or on purpose – in the various locations. Still there must always be means of synchronizing and consolidate the information from different sources. This kind of synchronization and consolidation is today performed ad-hoc, with poor tool support. By means of the EAXML format, extended to support versioning and configuration management, such consolidation will get proper tool support.

3 EATOP and ARTOP

EATOP is an open-source project which has the purpose of providing an Eclipse-based implementation of the EAST-ADL standard. EAST-ADL is a domain specific architecture description used in the automotive industry. One important feature of EATOP is ability to give access to an EMF-based version of the EAST-ADL meta-model. It does this by serializing and deserializing an EAXML file and presenting the result in the EAST-ADL Explorer.

ARTOP is similar to the EATOP project with the distinct difference that it focuses on the AUTOSAR domain instead. Compared to EAST-ADL it does not have such a high-level representation of a system but has a more detailed description level.

3.1 TableView Plug-in

It is possible to get details about each object in the EAST-ADL model by using the Properties View. This can be very helpful whilst either trying to understand the model or looking for problems and errors. However, it can be difficult to compare properties of many objects at the same time. Currently, the way to do this is to look at one object and then look at the other and try to discern the difference between them. Due to this a way to look at the properties in a table was desired and a prototype has been developed.

The plug-in has support for multiple selection and groups properties of the objects together in columns. Each row is thus represented by one of the selected objects in the EAST-ADL Explorer. As can be seen in Figure 1, several objects have been selected and different properties are presented in the Table View to the right. The Table View does not simply take the standard properties of the selected objects from the Properties View but specifies which properties each object should provide to the table. This has the advantage of filtering out only those properties which might be advantageous to compare and not cluttering the interface with unnecessary information.

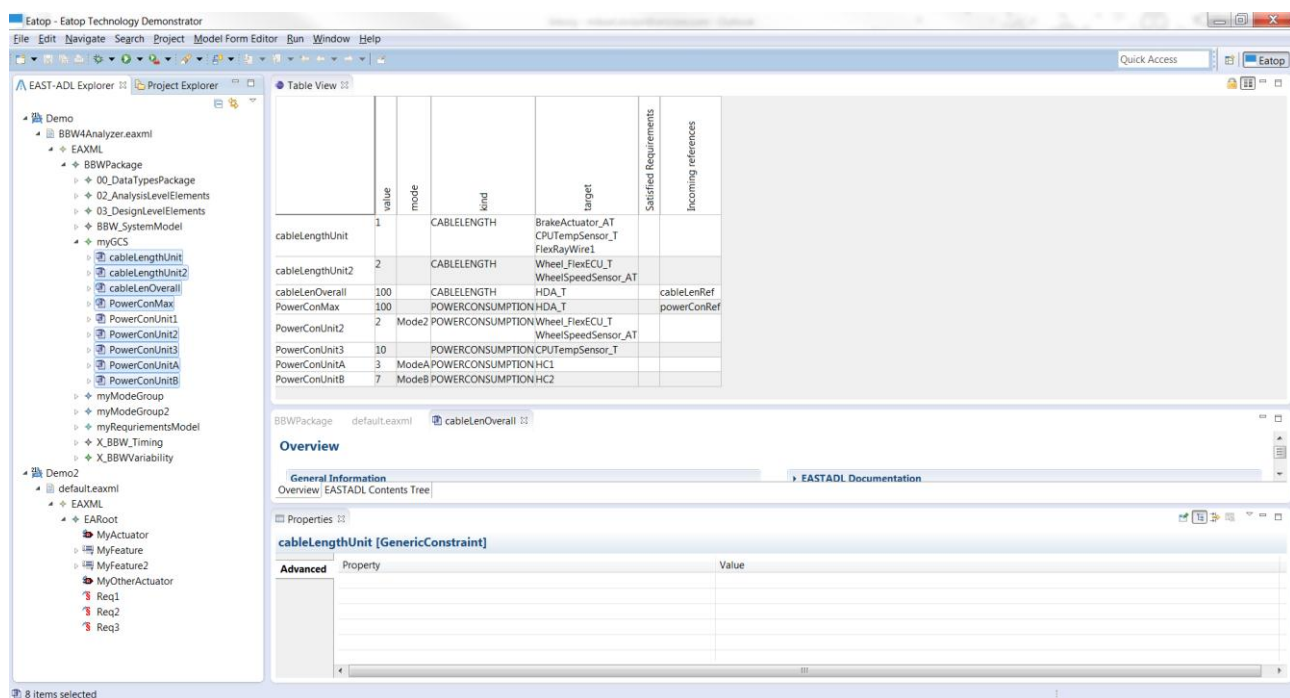


Figure 1, multi-select support in the TableView.

Furthermore, more advanced properties can be viewed in the table such as the “Incoming references” and “Satisfied Requirements” - columns seen in Figure 2. The TableView is still a prototype and a large development area of the TableView would be to increase the number of ways to access the information stored in the objects. It is currently possible to edit single elements in the table which is then propagated to the object in the model.

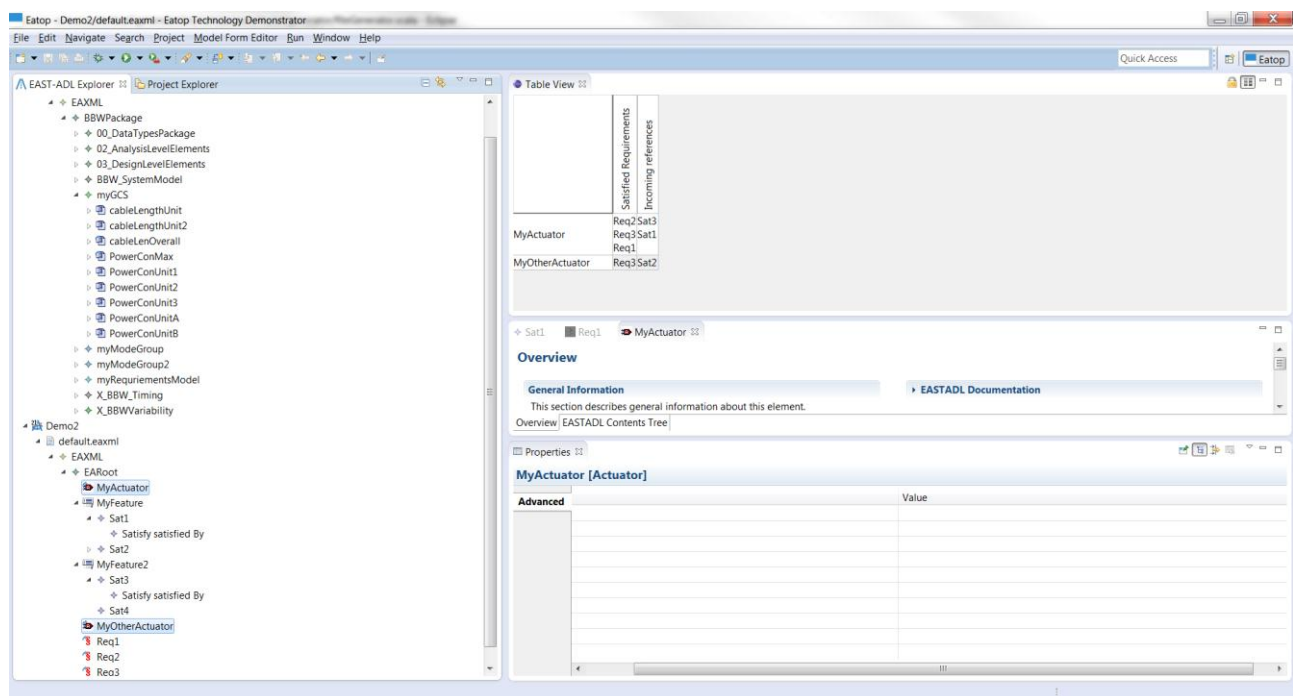


Figure 2, special properties shown in the table.

3.2 Create Connector Plug-in

When working with models in the EAST-ADL Explorer a common process is to create connectors between ports. This can be done through the explorer by creating a connector and manually linking the ports it should be connected between. It can be a rather tedious and time consuming process. Therefore, it is suggested to create a plug-in for EATOP which can assist the user when creating such connectors.

The concept focuses on aiding the user by visually presenting the ports which should be connected in a logical way. The trigger point should be the component prototype in order for the correct context can be used to check for validation. Figure 3 and Figure 4 demonstrates a mocked example of how it might function. As can be seen on the left side, when triggering the plug-in it sorts all the possible ports associated with the prototype which are possible candidates for creating the connector. Furthermore, when selecting one port the plug-in makes basic validation checks for possible matching candidates for the port. Validation should be kept rather simple, confirming only direction and client / server relations.

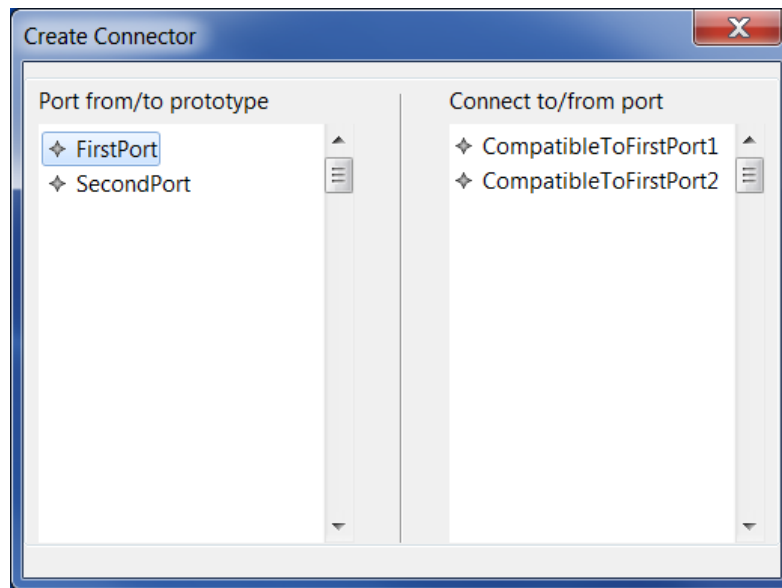


Figure 3, selecting the first port

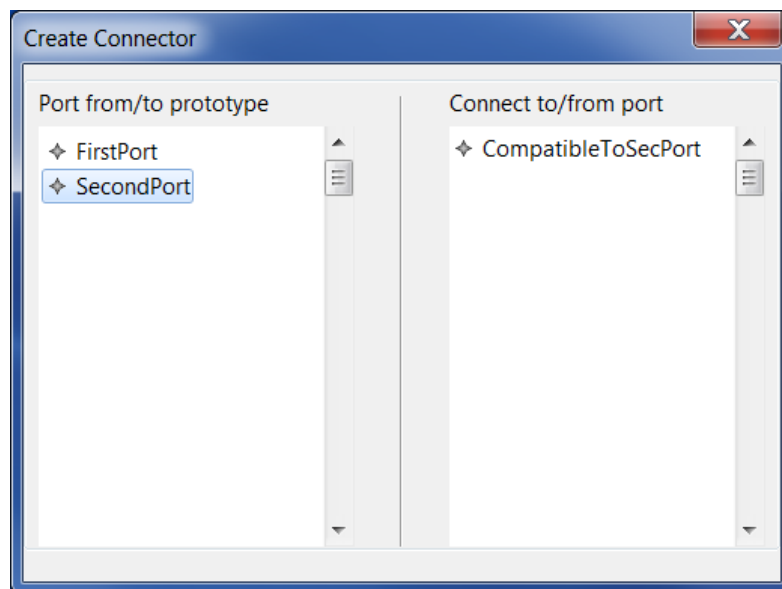


Figure 4, selecting the second port

After confirmation from the user the connector is created and the user should be informed about the result in an appropriate way. It is believed that a feature with this ability might increase the development process of users working with the EATOP tool environment.

3.3 Error Model Synthesis

Error propagation modelling is the basis for failure modes and effects analysis (FMEA) and fault tree analysis (FTA). EAST-ADL supports the structural definition of propagation models, and the propagation logic can be defined using different notations, depending on which kind of analysis and which tool is intended.

3.3.1 Description

The Error Model Synthesis plugin takes a nominal model representing the functionality under development. It then automatically creates an error propagation model that is structurally identical to the nominal model. On this basis, semi-automatic manipulations can be made to get an error model that reflects how errors are assumed to propagate.

The manipulations that are supported are

- **Combining Ports**
2 or more ports in the nominal model may have an equivalent meaning from a fault propagation viewpoint. To make the propagation model clearer, these ports can be automatically combined, and the plugin manages its references to the target port in the nominal model and the connectors.
- **Combining Prototypes (sub-models)**
2 or more prototypes (sub models) in an error model type may be combined to one, if it is deemed that this pair or group of functions can be represented by a common error model. A set of selected prototypes can be automatically combined, and the plugin manages its references to the target functions in the nominal model. The ports of the collapsed set will appear on the new, combined, prototype and the connectors are maintained.

Once an error propagation model is completed, the plugin is able to invoke an analysis tool, HiP-HOPS.

4 SystemWeaver

SystemWeaver is a generic collaborative system modeling platform developed and sold by Systemite AB.

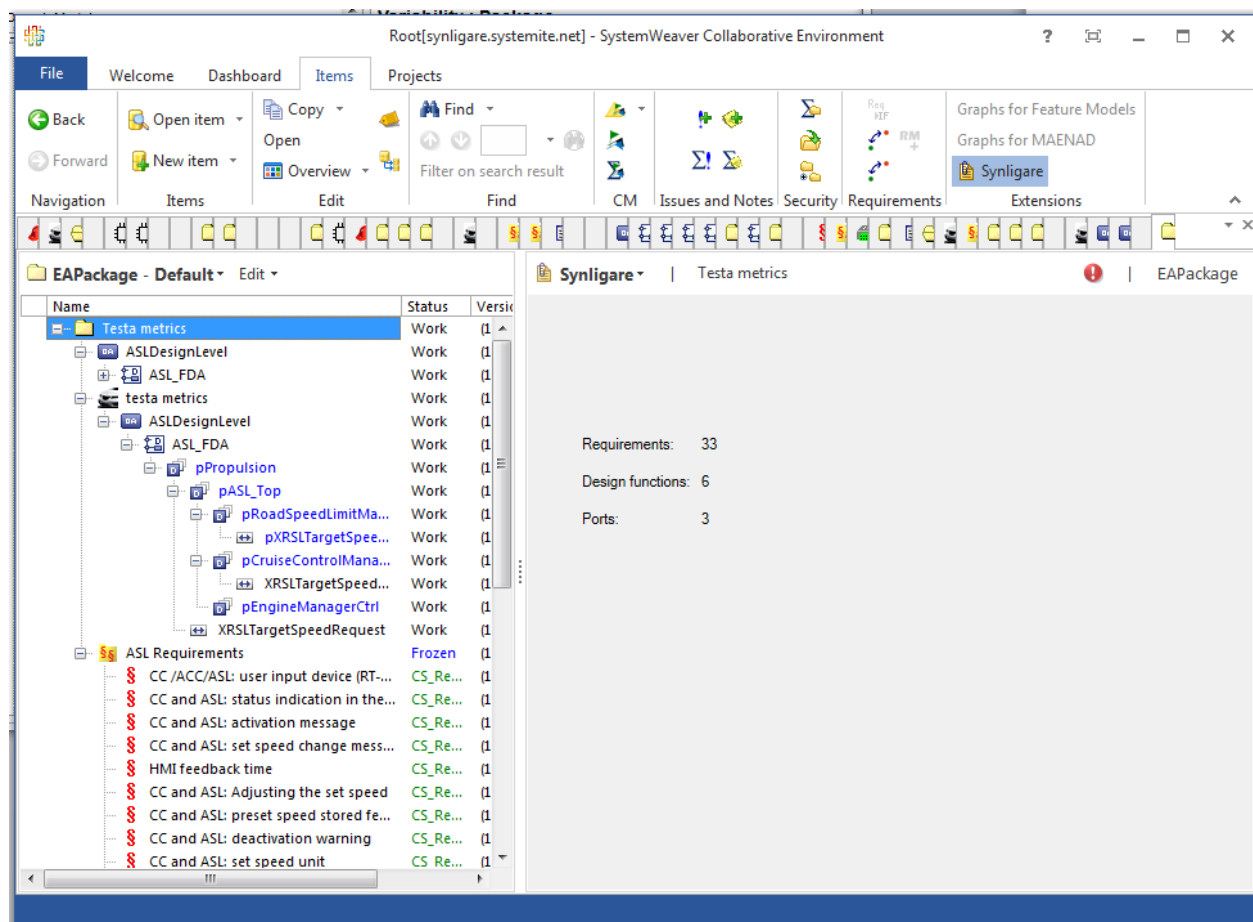
The specific functionality of SystemWeaver is defined by the installed meta model and view extensions.

The Meta model in SystemWeaver supports the core part of EAST-ADL, and extensions like the Requirements and Dependability packages.

For the Synligare project Systemite hosts a server at synligare.systemite.net, and a suitable SystemWeaver client, along with all extensions to support special views developed for the Synligare project, can be installed by using the ClickOnce installation script available at <http://apps.systemite.se/Synligare/setup.exe>.

4.1 Metrics Extension

The figure below shows the “0” iteration of the metrics view implemented in SystemWeaver, counting the number of unique requirements, DesignFunctionPrototypes and Ports.



5 EnterpriseArchitect

The Enterprise Architect UML tool is a widely used, low cost UML editor. With an EAST-ADL profile, it can also be used as an EAST-ADL editor.

5.1 Enterprise Architect EAXML Exchange

The Enterprise Architect EAST-ADL Exchange tool allows EAXML files to be imported and exported to Enterprise Architect.

5.1.1 Description

The Enterprise Architect EAST-ADL Exchange tool is implemented as a stand alone application that reads and writes EAXML files. The API of Enterprise Architect is used to create and read elements in an EAST-ADL model in Enterprise Architect.

The import and export from EnterpriseArchitect has been prototyped in a first step.

6 Summary

This deliverable has documented the tooling prototypes developed in the Synligare project.

7 References

- [1] EATOP Eclipse Open Source Project: EAST-ADL Tool platform.
<http://www.eclipse.org/eatop>
- [2] Sparx Systems Inc: Enterprise Architect. www.sparxsystems.com
- [3] Synligare Consortium: Synligare Deliverable D1.1 Needs Identification.
<http://www.synligare.eu/>
- [4] Synligare Consortium: Synligare Deliverable D2.1 Modeling Concepts and Methods.
<http://www.synligare.eu/>
- [5] Systemite AB: SystemWeaver. <http://www.systemite.se>